Lannet

Normalization:

First Normal Form (1NF) Definition: A relation is in First Normal Form (1NF) if all attributes are atomic (indivisible) and there are no repeating groups or multi-valued attributes.

No table stores arrays, comma-separated lists, or repeating column groups (e.g. phone1, phone2). Every column holds one atomic value per row. Fix: No changes required — all values are already atomic.

No table uses repeating group columns. Related multi-valued data (e.g. files per milestone, notifications per user) are already stored in separate child tables with FKs.

All 13 tables satisfy 1NF.

2. Second Normal Form (2NF)

Definition: A relation is in Second Normal Form (2NF) if it is already in 1NF and every non-key attribute is fully dependent on the entire primary key, not just part of it

Partial dependencies can only occur when a table has a composite primary key. Because every table in this schema uses a single-column surrogate integer PK (AUTO_INCREMENT), partial dependencies are structurally impossible — there is no 'part' of the key to partially depend on

All 13 tables satisfy 2NF.

3. Third Normal Form (3NF)

Definition: A relation is in Third Normal Form (3NF) if it is already in 2NF and has no transitive dependencies (non-key attributes depending on other non-key attributes)

1. In ESCROW, attributes client_id and worker_id stores redundant data

   FD:

   escrow_id → milestone_id

   milestone_id → project_id

   project_id → client_id, gig_worker_id

   therefore, escrow_id → client_id and worker_id are transitive dependencies

   fix: remove client_id and worker_id from ESCROW.

   Retrieve them via JOIN: ESCROW → MILESTONES → PROJECTS when needed.

2. In MILESTONES, worker_id was transitively derivable through project_id.
   FD:
   milestone_id → project_id
   project_id → gig_worker_id
   therefore, milestone_id → worker_id is a transitive dependency

   Fix: drop worker_id from MILESTONES.
   Worker is always retrievable via: MILESTONES.project_id → PROJECTS.gig_worker_id.

   Now the db is in 3 NF

4.BCNF

Definition: A relation is in BCNF if it is already in 3NF and for every functional dependency X → Y, X must be a superkey.

After applying 3NF fixes, we checked all tables for non-superkey determinants. Three tables have more than one candidate key:

USERS — candidate keys: user_id, email (UNIQUE) WALLETS — candidate keys: wallet_id, user_id (UNIQUE) ESCROW — candidate keys: escrow_id, milestone_id (UNIQUE)

Functional Dependencies:

user_id → email and email → user_id (both are superkeys)

wallet_id → user_id and user_id → wallet_id (both are superkeys)

escrow_id → milestone_id and milestone_id → escrow_id (both are superkeys)

Fix: No changes required. In all three cases both candidate keys are superkeys, so no BCNF violation exists. All remaining tables have a single candidate key (their surrogate PK), making BCNF violations impossible for those tables.

Result: All 13 tables satisfy BCNF. Since every table uses a single-column surrogate primary key and the only tables with multiple candidate keys (USERS, WALLETS, ESCROW) have all candidate keys acting as superkeys, no determinant exists that is not a superkey. Schema fully satisfies BCNF.